

Module 1

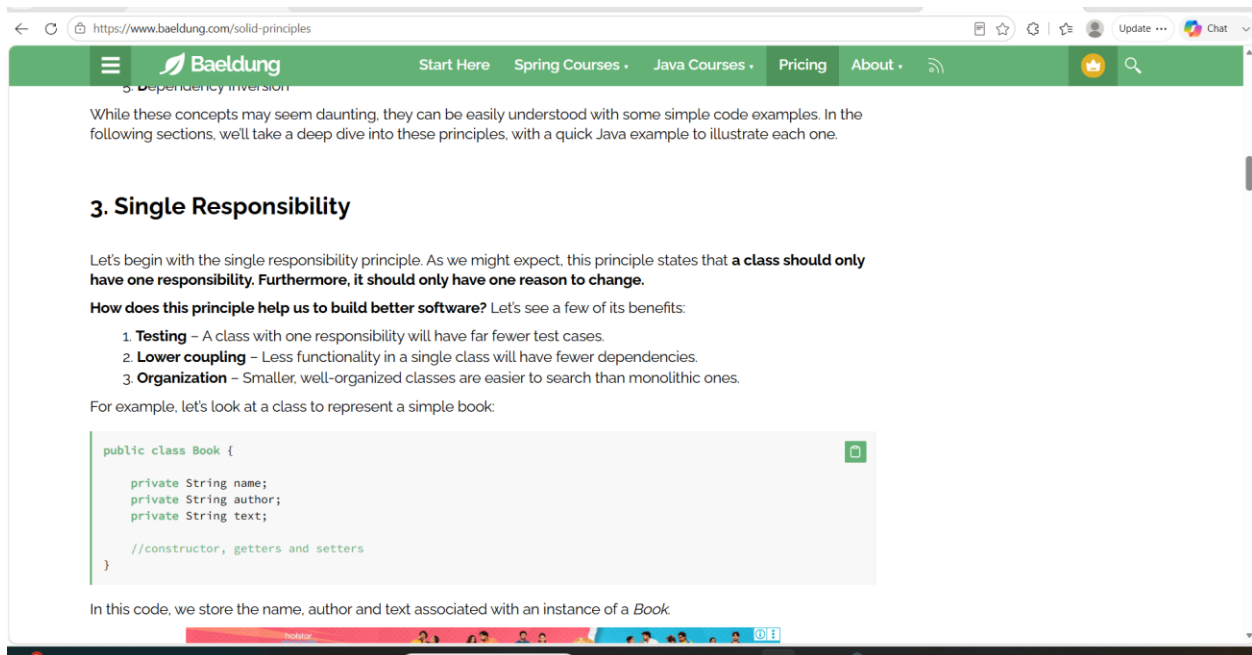
Design Patterns and Principles

Report on Self Learning

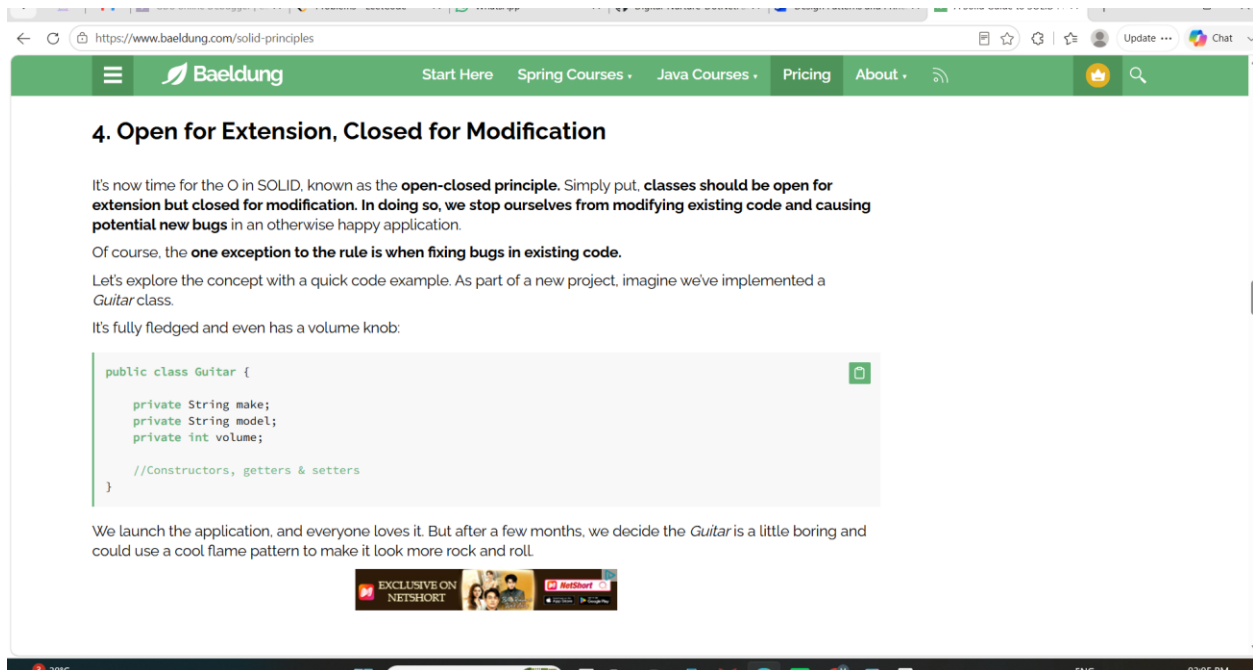
I Successfully completed Module 1-Design Patterns and Principles and gained a solid understanding of software design concepts and best practices. This module has given me an understanding of the need for designing software that is maintainable, scalable, reusable, and extensible. Understood the importance of software design principles in OOPS and their role in creating effective and reliable software solutions.

In this I covered SOLID Principles of Object Oriented Programming and an understood what they are and how they can be used. I learned about the below topics:

- Single Responsibility Principle (SRP) – Ensuring that a class has only one responsibility.



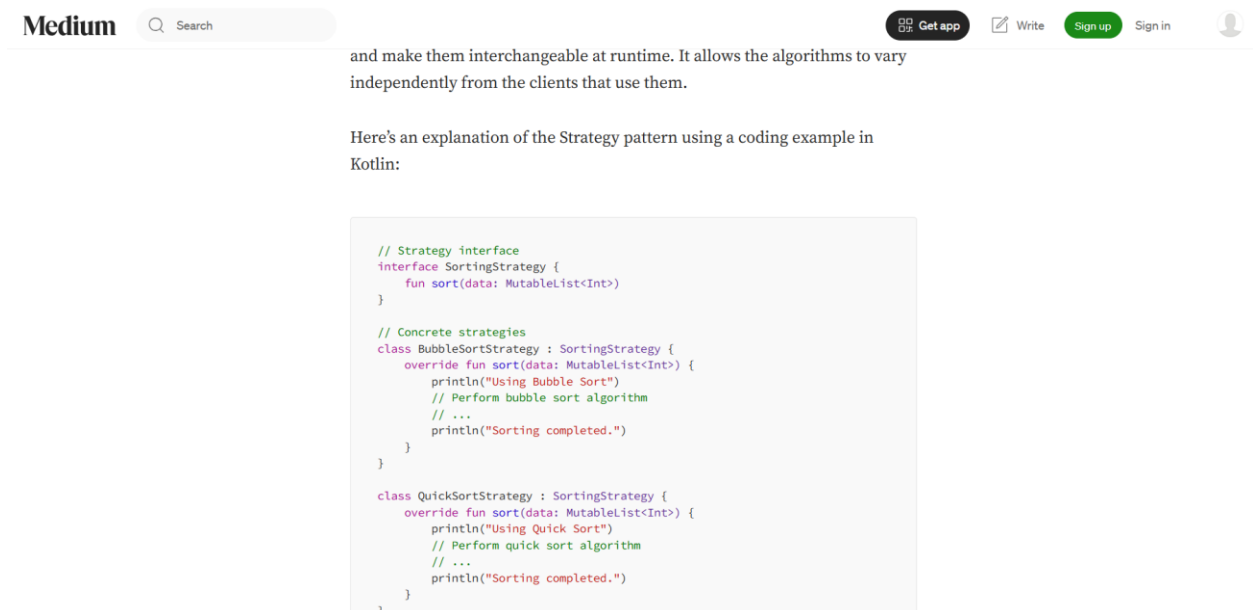
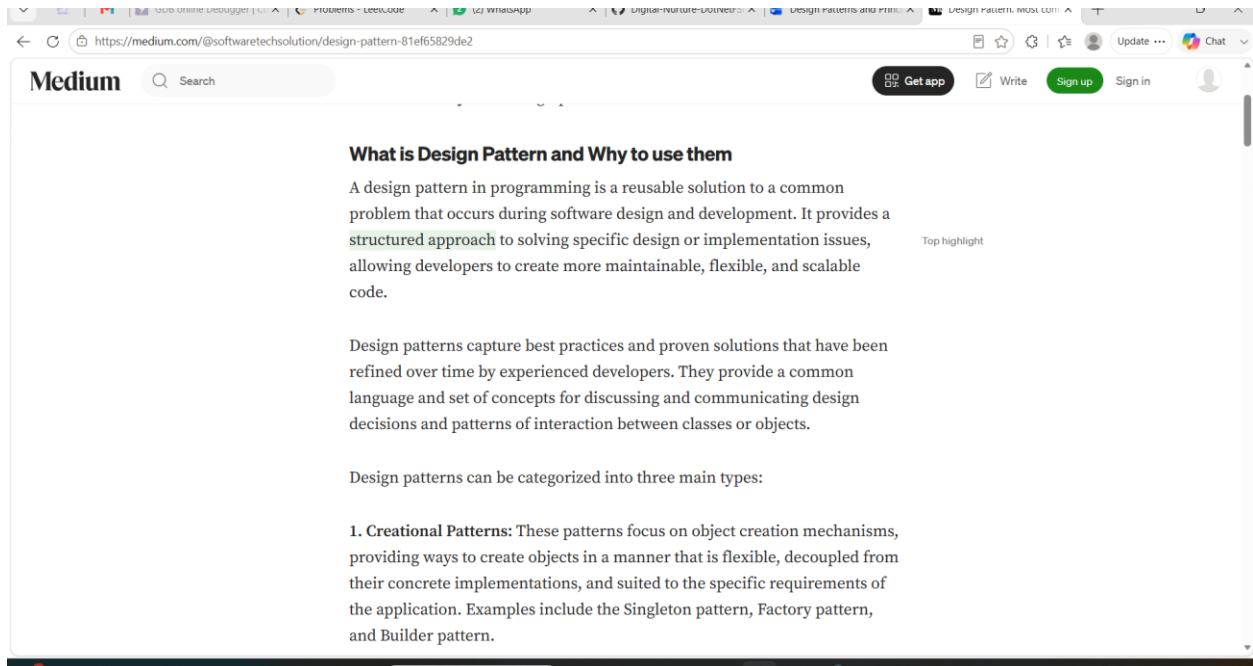
- Open-Closed Principle (OCP) – Design objects so they are open for extension, but closed for modification.



- Liskov Substitution Principle (LSP) – Making sure that derived classes can be substituted for base classes without compromising the correctness of the program.
- Interface Segregation Principle (ISP) – Design small, specific interfaces instead of large, general interfaces.
- Dependency Inversion Principle (DIP) – Depend on abstractions rather than concrete types.

I also looked at some of the Gang of Four (GoF) design patterns that are commonly used and how they offer solutions to common software design problems that can be reused. Some of the patterns I learned were:

- Creational Patterns – Singleton Pattern, Factory Method Pattern, Builder Pattern.
- Structural Patterns – Adapter Pattern, Decorator Pattern, Proxy Pattern.
- Behavioral Patterns – Observer Pattern, Strategy Pattern, Command Pattern.
- Model-View-Controller (MVC) and Dependency Injection architectural patterns.



I learned to recognize software design issues and apply appropriate software design patterns to solve them by engaging in hands-on activities and practical examples. I gained insight into how these principles and patterns can increase code reusability, minimize duplication, make software more maintainable, and create a scalable software architecture. In summary, this module deepened my knowledge of Object Oriented Design and gave me the opportunity to build clean, efficient software solutions.

